you when a memory leak pattern is detected, usually inferred if the JVM heap usage metric keeps increasing over time. If you don't catch the memory leak in advance, the application will throw the dreaded `OutOfMemoryError` error and crash.

Once you suspect an application might suffer from a memory leak, you must find out which objects are held in memory and block the garbage collection. There are different ways to proceed with finding problematic objects. For example, you could enable the Java Flight Recorder or attach a profiler like jProfiler to the running application. Another way is to take a snapshot of all the Java objects in the JVM heap memory (a *heap dump*), and analyze it with a specialized tool to find the root cause of the memory leak.

Spring Boot Actuator provides a convenient endpoint (`/actuator/heapdump`) that you can call to generate a heap dump. Let's see that in action. Go to your Catalog Service project (catalog-service), open the application.yml file, and configure Actuator to expose the `heapdump` endpoint.

Listing 13.27 Exposing the `heapdump` Actuator endpoint

```
management:
  endpoints:
    web:
      exposure:
        include: flyway, health, heapdump, info, prometheus     ❶
```

❶ Adds heapdump to the list of Actuator endpoints to be exposed over HTTP

Next, build and run Catalog Service (`./gradlew bootRun`). Finally, invoke the `heapdump` endpoint:

```
$ http --download :9001/actuator/heapdump
```